

A Deep Dive into Agoda's Generic Reconciliation Platform | by Agoda Engineering & Design | Medium

By Agoda Engineering

A Deep Dive into Agoda's Generic Reconciliation Platform by Songkun Viriyavaree

Introduction At Agoda, managing intricate financial transactions is a critical part of our operations. These transactions involve a diverse group of participants, ranging from customers and hotels to banks, credit card companies, and other online travel agencies. To maintain the highest level of precision and integrity in these transactions, Agoda has a specific system. The main goal of this system is to find and resolve any discrepancies or anomalies in our transactions. This article will explore how we approach the creation of a reconciliation system. Our goal is to build a system that is both scalable and reliable. We plan to achieve this by effectively utilizing Spark.

Why is a Generic Reconciliation Platform Essential for Finance? Currently, Agoda's finance team uses various reconciliation systems. These systems are built with different frameworks and programming languages. This variety leads to a lack of standardization in reconciliation processes, complicating system maintenance. Moreover, developing a new reconciliation system for each new business case is time-consuming. It can take several weeks, mainly because there isn't an existing platform to build upon.

What Generic Reconciliation Offers

Simplifying Maintenance: Our new reconciliation platform keeps all reconciliation systems in a single spot. This centralization simplifies maintenance, reducing both workload and costs. It also fosters resource sharing among developers and encourages consistent terminology. For example, updating a library can now be done in one place.

Scalability Through Spark: Our platform uses Spark as its data processing engine. This allows for efficient handling of large data volumes, enhancing scalability.

Updates Made Easy: Changes to the system are implemented in this central repository. This simplifies updates and eliminates the need for multiple repositories.

Robust Monitoring System: Our platform diligently watches over system health and product health. For system health, we use Slack alerts for real-time monitoring of the reconciliation process. This ensures immediate identification of issues. We also track input and output transaction data on our Grafana dashboard for anomaly detection and reporting. Product health monitoring allows users to set key performance indicators (KPIs) like unreconciled percentages and reconciled amounts. These are stored in a big data table, enabling easy dashboard creation for intelligent decision-making by business teams.

Integration with Finance System The Generic Reconciliation Platform integrates with the finance system to reconcile data from two distinct sources and to generate reports to be recorded on big data tables. This multifunctional platform supports several types of reconciliation, including account receivable and partner reconciliation.

In the diagram above, we illustrate an example of partner reconciliation. On the left, it shows data gathered from various partners. This data is downloaded using a Spark application, called 'Downloader'. The Downloader fetches files from multiple sources, like SFTP, and stores them in big data tables. After the data lands in these big data tables, another program, 'Normalizer', transforms this data, assimilating information into a standardized table, thereby making it accessible to the reconciliation platform. On the right side of the diagram, we see Agoda data, which has been regularized from various internal data sources. Once both sets of data are prepared and normalized, we proceed to reconcile and categorize the data into different classifications — categorizing buckets for each data set, making it easily accessible for various applications. This reconciliation result is then utilized by different applications, like Metabase, for dashboard and report generation; finance reporter generator for building enterprise planning reports; and file uploader for transferring data back to the partner via email or SFTP.

Reconciliation Ideology The core of reconciliation is identifying a 'key' and a

corresponding 'value.' The 'key' determines if two entities are comparable. In partner reconciliation, for instance, the `partner_id` from both Agoda and the partner must align for any comparison to proceed. If the 'keys' of corresponding transactions match, we then specify the 'value' to be compared. In the case of partner reconciliation, the 'value' is a field termed `partner_amount`, which represents the amount payable to the partner. Only after comparison can the data be confirmed and moved to the reconciled bucket.

Introducing Generic Reconciliation

The functionality of the Generic Reconciliation Platform can be broken down into three major components. First is the 'Data Preparation' phase. In this phase, we collect and prepare data from a variety of sources, a crucial step for successful reconciliation. Next, the 'Reconciliation' phase is where we compare and align data from the two aforementioned sources. Lastly, we have the 'Post Reconciliation' phase, where data validation occurs, and Key Performance Indicators (KPIs) are developed for usage on the dashboard. Within each of these components, processes are broken down into distinct processors. Let's take the 'Data Preparation' segment as an example. It's segmented into three different processors: the 'Prepare Processor,' the 'Normalize Processor,' and the 'Exclude Processor.' Each type of reconciliation requires its own unique strategy. These strategies guide the processors on how to execute their tasks effectively. We have also created an interface that allows each reconciliation process to define its own strategies, enabling a tailored reconciliation approach.

Data Preparation

The preparation process is divided into three distinct steps:

- 1. Fetching Data:** This step involves collecting new data, recovering previously corrupt data, and fixing corrupted data.
- 2. Data Deduplication:** In this step, we deduplicate the data. To ensure each piece of data is unique, we use a 'deduplicate_key.' This key is created by combining different columns.
- 3. Data Validation:** The final step is about validating the data. We achieve this by following a specific set of rules that we have established.

The normalize process

The normalize process is divided into two steps:

- 1. Combining New and Unreconciled Data:** This involves merging data that was unreconciled from the previous day with new data.
- 2. Data Transformation:** In this step, we transform data into the new format and define `reconciliation_amount` columns based on a set of rules.

The exclude process

The exclude process is divided into two steps:

- 1. Conditional Exclude:** In this step, data is excluded based on a set of defined rules.
- 2. Manual Exclude:** In this process, users can enter the specific deduplication keys corresponding to the transactions they wish to eliminate. These keys are then incorporated into the manual exclusion action. This action subsequently correlates these keys with the incoming data, effectively erasing the selected transactions from the system.

Reconciliation

The reconciliation phase is the core of the entire system. It processes unreconciled transactions and attempts to reconcile them over several passes or conditions based on configuration. For every pass, we can reconcile based on key and reconciliation columns. Following the reconciliation procedure, there might still be some transactions left unreconciled. In these cases, we have the ability to force reconcile them using a specific set of rules or logic. In the reconciliation process, it's possible to define the key and the value based on certain rules. Here's an example: If we're dealing with reconciliation with Partner A, we may decide to reconcile value solely based on the `partner_amount`. However, if the partner is B, we might choose to reconcile values using both the `partner_point` and `partner_amount` columns. This is just one insight into how we can specifically tailor configurations for values depending on our respective partners. There are more scenarios similar to this that use conditions such as `partner_id` to configure reconciliation keys. In the reconciliation process, it's not uncommon for a single transaction to go through multiple rounds of reconciliation. Consider a booking with multiple transactions. In the first pass, we might try to reconcile a transaction using its unique `transaction_id`. If this first attempt fails to reconcile the transaction, it then proceeds to a second pass. In this next stage, the `booking_id` is used as the key identifier for reconciliation. This two-step approach ensures that we have an alternative method to reconcile a transaction when the first attempt isn't successful. Another part of the process is the force reconciliation part

where the transactions are moved to reconcile buckets based on these two steps:

Conditional Force Reconciled: This step forcefully reconciles data based on a set of defined rules.

2. Manual Exclude: In this process, users can enter the specific keys, like `partner_id`, that correspond to the transactions they intend to remove. After entering these keys, they are integrated into a manual force reconciliation action. This action then aligns these specific keys with incoming data, effectively moving the selected transactions to the reconcile bucket.

CI/CD Pipeline We utilize three types of tests in the CI/CD pipeline, namely the unit test, integration test, and shadow test. The purpose and functions of these tests are as follows:

- 1. Unit Test:** This aims to examine individual processors and services, ensuring their independent operability.
- 2. Integration Test:** It focuses on evaluating the comprehensive reconciliation process and validating the outcomes from the output tables.
- 3. Shadow Test:** This method involves operating modified code, for example, Merge Request (MR) changes, on the production data. The goal is to compare outcomes and assure both accuracy and validity. There are three steps in running the shadow tests.

Setup Context: This focuses on setting up the context such as branch information and process data for shadow test.

Run Recon: This focuses on running the reconciliation using production data with the modified logic.

Compare Result: This involves updating the notes in GitLab by sharing a comparative analysis between the production data/schema and the test result data. Some important aspects of shadow testing are:

- Shadow tests are triggered manually. This is because we want to ensure that the pull request is ready before the shadow test is triggered since running the shadow test is quite expensive since it involves production data. The shadow test outcomes aren't a mandatory requirement for merging the PR. For instance, if a logical update is made, it's perfectly normal to expect variations in the output data compared to the production data.

Monitoring and Alerting The Generic Reconciliation Platform at Agoda relies on three services for monitoring and alerting purposes. First, we utilize Metabase to create dashboards that track business and product Key Performance Indicators (KPIs). Second, Grafana is employed for monitoring job performance and quality. Lastly, Slack serves as our notification system, alerting us when a job fails or passes. We use Metabase to design numerous graphs, effectively enabling both business and product managers to assess the performance of our reconciliation system. For instance, by examining the total unreconciled amount in USD as a percentage, product managers can make informed decisions on whether further data investigation is necessary. Once the reconciliation process concludes, we receive a Slack alert. This alert indicates the success or failure of the reconciliation — a green sidebar denotes success, while a red sidebar signifies failure along with failure information. The Slack alert doesn't just carry the state of the job but also links to vital resources like KPI, cluster log, and the spark UI link. Additionally, we have developed a comprehensive Grafana dashboard for the platform. It tracks the volume of input transactions and their distribution across different buckets. Additionally, we created a second dashboard to monitor the status and duration of every reconciliation process.

Conclusion Reconciliation plays a pivotal role in ensuring the accuracy and validity of our financial system. We are developing a robust, generic reconciliation system capable of managing millions of daily transactions. Our system leverages the Spark data processing framework to enhance its efficiency. The beauty of this advanced reconciliation setup is its scalability, maintainability, and intrinsic monitoring capabilities, all of which are crucial for dealing with different types of reconciliation.